

Product Requirement Document (PRD)

Project Title

CarCare API — Car Maintenance Center Management Backend

1. Business Problem

Traditional automotive service centers rely on fragmented communication channels (manual logs, phone bookings, uncoordinated spreadsheets), leading to operational inefficiencies:

- **Booking Collisions & Double Booking:** Vehicles are scheduled for overlapping service slots, creating shop-floor bottlenecks.
- **Lack of Status Visibility:** Customers have no real-time insight into whether their car is queued, under repair, or ready for pickup.
- **Pricing Instability:** Updating a service's base price retroactively alters historical booking records if price snapshots are not stored.
- **Manual Operational Overhead:** Service staff must manually monitor unconfirmed bookings and remind customers, leading to stale appointments that block active calendar slots.

System Goal: Deliver a secure, high-performance RESTful Web API backend that automates the appointment lifecycle, enforces strict domain business rules, isolates access by user roles, and offloads heavy notification and maintenance tasks to asynchronous background jobs.

2. Feature Overview & Scope

2.1 Authentication & Profile Management

- Customer registration and secure JWT authentication.
- Role-based token issuance (Customer, Staff, Manager).
- Token claim extraction (sub, email, role) to prevent identity tampering.

2.2 Vehicle Fleet Management

- Vehicle registration linked directly to authenticated customer accounts.
- Profile management and historical vehicle service queries.

2.3 Service Catalog Administration

- Dynamic service catalog management (oil change, brake inspection, engine diagnostics).
- Activation/deactivation of services without destroying historical data.
- Managerial controls for base price and estimated duration updates.

2.4 Appointment Lifecycle

- Multi-service appointment booking with historical price snapshotting (UnitPrice).
- State Machine transitions: **Requested** -> **Confirmed** -> **InService** -> **Completed**, along with terminal **Cancelled** and **NoShow** states.
- Vehicle conflict verification prior to booking confirmation.

2.5 Background Jobs & Automated Operations

- Staff notification upon appointment creation.
- Customer notifications upon status transition.
- Cancel unconfirmed or stale appointments after 24 hours.

3. Actors & User Roles

Actor	Description	Authorization Scope
Customer	Registered car owner using the platform to schedule and track vehicle service appointments.	Manage own vehicles, view active services, book appointments, view own appointments, cancel eligible bookings.
Staff	Maintenance center technician or service desk representative managing daily garage operations.	View overall garage schedule, filter appointments, transition appointment states (Confirmed, InService, Completed, NoShow).
Manager	Center administrator overseeing business configurations and operational performance.	Full system permissions: service catalog creation or updates, service activation or deactivation, monitoring, operational reports.

4. Business Rules

- **BR-01 (Authentication Mandatory):** The platform operates on a secure-by-default access control policy. System capabilities and data are restricted strictly to authenticated users, with unauthenticated access is limited to registration, login, and viewing the active public service catalog.
- **BR-02 (Vehicle Ownership Validation):** A customer can only create or modify appointments using a **VehicleId** explicitly owned by their authenticated **CustomerId**.
- **BR-03 (Service Active Constraint):** Appointments can only be booked with services where **IsActive = true**.
- **BR-04 (Future Scheduling Constraint):** All new appointment dates must be set strictly in the future relative to server UTC time (**AppointmentDate > DateTime.UtcNow**).
- **BR-05 (Vehicle Booking Conflict Prevention):** Appointment duration is mathematically defined as:

$$\text{TotalDuration} = \sum \text{Service.DurationMinutes}$$

Appointment timeframe: Start = AppointmentDate

End = Start + TotalDuration

A vehicle cannot have more than one active (Requested,Confirmed,InService) appointment in an overlapping time window.

A conflict occurs if: $(ExistingStart < NewEnd) \wedge (ExistingEnd > NewStart)$.

- **BR-06 (Customer Self-Cancellation Only):** A customer can only cancel appointments associated with their own account.
- **BR-07 (In-Service Cancellation Lock):** An appointment cannot be cancelled by a customer once its status has reached InService or Completed.
- **BR-08 (Strict Status Transition Matrix):** Status changes must strictly follow forward-only rules:
Requested ---> Confirmed | Cancelled
Confirmed ---> InService | Cancelled
InService ---> Completed| No Show
Completed ---> [Terminal State - No Changes Allowed]
Cancelled ---> [Terminal State - No Changes Allowed]
- **BR-09 (Staff/Manager Role Elevation):** Only users with Staff or Manager roles are authorized to invoke status transition endpoints ([PUT /api/appointments/{id}/status](#)).
- **BR-10 (Price Snapshotting):** When an appointment is created, the service price must be recorded into AppointmentService.UnitPrice. Historical booking prices remain unchanged if the master Service.BasePrice is updated later.
- **BR-11 (Deactivation Immunity):** Deactivating a service (**IsActive = false**) prevents new bookings from using it but does not invalidate existing historical appointments containing that service.
- **BR-12 (Claim-Based Ownership Enforcement):** API endpoints must infer user identity directly from JWT claims (sub / UserId) rather than relying on request body payloads, mitigating insecure direct object reference (IDOR) attacks.
- **BR-13 (Public Registration Restriction):** Public registration (POST /api/auth/register) strictly provisions Customer accounts. Staff and Manager accounts cannot be self-selected and are created via administrative seeding processes.
- **BR-14 (Atomic Conflict Prevention):** Appointment creation and overlap conflict checking must be executed within a database transaction using strict concurrency locking mechanisms. This ensures atomicity and prevents race-condition double bookings, maintaining data integrity during high-frequency scheduling.
- **BR-15 (Stale Appointment Cleanup):** The system must automatically identify stale appointments, defined as those with a **Status = Requested** and a **CreatedAt** timestamp older than 24 hours. A recurring Hangfire background job will run hourly to detect these records and update their status to **Cancelled**.

5. Data Model

5.1 Entities & Attributes

ApplicationUser (ASP.NET Core Identity Entity)

Field	Type	Constraints	Description
Id	Guid	Primary Key	Unique identity user ID
Email	string	Max Length: 100, Required, Unique	Security login email
PhoneNumber	string	Max Length: 20, Required	Contact phone number

Customer (Domain Entity)

Field	Type	Constraints	Description
Id	Guid	Primary Key	Unique domain customer ID
UserId	string / Guid	Foreign Key (ApplicationUser), Required	Reference to Identity user account
Name	string	Max Length: 100, Required	Customer full name
CreatedAt	DateTime	Required	Profile creation UTC timestamp

Vehicle

Field	Type	Constraints	Description
Id	Guid	Primary Key	Unique vehicle identifier

CustomerId	Guid	Foreign Key (Customer), Required	Owner reference
PlateNumber	string	Max Length: 20, Required	Vehicle license plate number
Vin	string	Max Length: 17, Required, Unique	Vehicle Identification Number
Make	string	Max Length: 50, Required	Manufacturer (e.g., Toyota)
Model	string	Max Length: 50, Required	Model name (e.g., Corolla)
Year	int	Range: [1950, CurrentYear + 1]	Manufacturing year
CreatedAt	DateTime	Required	Record creation UTC timestamp

Service

Field	Type	Constraints	Description
Id	Guid	Primary Key	Unique service identifier
Name	string	Max Length: 100, Required	Service title
Description	string	Max Length: 500	Detailed service scope

DurationMinutes	int	Range: \$> 0\$	Estimated completion time in minutes
BasePrice	decimal	Precision: (18,2), \$\\ge 0\$	Default catalog price
IsActive	bool	Required, Default: true	Active status flag for new bookings
CreatedAt	DateTime	Required	Record creation UTC timestamp

Appointment

Field	Type	Constraints	Description
Id	Guid	Primary Key	Unique appointment identifier
CustomerId	Guid	Foreign Key (Customer), Required	Customer reference
VehicleId	Guid	Foreign Key (Vehicle), Required	Assigned vehicle reference
AppointmentDate	DateTime	Required	Scheduled service time (UTC)
Status	Enum	Required, Default: Requested	Current lifecycle state
Notes	string	Max Length: 1000, Optional	Customer notes/issues

			reported
CreatedAt	DateTime	Required	Record creation UTC timestamp
UpdatedAt	DateTime?	Optional	Last state transition UTC timestamp
CancelledAt	DateTime?	Optional	Cancellation UTC timestamp

AppointmentService (Join Table / Snapshot)

Field	Type	Constraints	Description
AppointmentId	Guid	Primary Key / FK (Appointment)	Parent appointment reference
ServiceId	Guid	Primary Key / FK (Service)	Included service reference
UnitPrice	decimal	Precision: (18,2), Required	Price snapshot at time of booking
DurationMinutes	int	Range: \$> 0\$	Estimated completion time in minutes

5.2 Entity Relationships

Customer(1) —< (N) Vehicle

Customer(1) —< (N) Appointment

Vehicle (1) —< (N) Appointment

Appointment (1) —< (N) AppointmentService (N) >— (1) Service

6. API Endpoints Specification

Module 1: Authentication (/api/auth)

POST /api/auth/register

- **Purpose:** Registers a new customer and creates the linked Customer entity.
- **Access:** Public
- **Request Body:**
JSON

```
{
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "phone": "01234567890",
  "password": "SecurePassword123!"
}
```
- **Responses:**
 - 201 Created: User and Customer profiles created successfully.
 - 400 Bad Request: Validation failure (e.g., weak password, invalid email format).
 - 409 Conflict: Email address is already registered.

POST /api/auth/login

- **Purpose:** Authenticates user credentials and generates a signed JWT.
- **Access:** Public
- **Request Body:**
JSON

```
{
  "email": "jane.doe@example.com",
  "password": "SecurePassword123!"
}
```
- **Responses:**
 - 200 OK: Returns Bearer access token, expiration, userId, and role.
 - 401 Unauthorized: Invalid credentials.

GET /api/auth/me

- **Purpose:** Retrieves identity and profile information for the authenticated user.
- **Access:** Authorized (Customer, Staff, Manager)
- **Responses:**
 - 200 OK: Profile data returned.
 - 401 Unauthorized: Missing or invalid JWT.

Module 2: Vehicles (/api/vehicles)

POST /api/vehicles

- **Purpose:** Registers a vehicle belonging to the calling customer.
- **Access:** Authorized (Customer)
- **Request Body:**
JSON

```
{
  "plateNumber": "DEF-5678",
  "vin": "1HGCR2F83HA000000",
  "make": "Honda",
  "model": "Accord",
  "year": 2021
}
```

- **Responses:**

- 201 Created: Vehicle registered.
- 400 Bad Request: Validation failure (e.g., invalid year or empty fields).
- 409 Conflict: VIN already exists in the system.

GET /api/vehicles/my

- **Purpose:** Returns all vehicles owned by the authenticated customer.
- **Access:** Authorized (Customer)
- **Responses:**
 - 200 OK: List of customer's vehicles.

GET /api/vehicles/{id}

- **Purpose:** Retrieves details for a specific vehicle.
- **Access:** Authorized (Customer, Staff, Manager)
- **Responses:**
 - 200 OK: Vehicle details.
 - 403 Forbidden: Customer attempting to access another user's vehicle (BR-02).
 - 404 Not Found: Vehicle ID does not exist.

GET /api/vehicles/{id}/service-history

- **Purpose:** Returns historical appointments and completed service logs for a vehicle.
- **Access:** Authorized (Customer for own vehicle, Staff, Manager).

Module 3: Services Catalog (/api/services)

GET /api/services

- **Purpose:** Lists all active services available for booking.
- **Access:** Public / Authenticated
- **Responses:**
 - 200 OK: Service list returned.

PUT /api/services/{id}

- **Purpose:** Updates catalog service attributes (name, description, duration, base price).
- **Access:** Authorized (Manager)

POST /api/services

- **Purpose:** Creates a new service offering in the catalog.
- **Access:** Authorized (Manager)
- **Request Body:**

```
JSON
{
  "name": "Full Synthetic Oil Change",
  "description": "Includes filter replacement and fluid top-off.",
}
```

```
"durationMinutes": 45,  
"basePrice": 85.00  
}
```

- **Responses:**
 - 201 Created: Service created.
 - 400 Bad Request: Invalid price/duration parameters.
 - 403 Forbidden: Caller lacks Manager role.

PATCH /api/services/{id}/deactivate

- **Purpose:** Deactivates a service to prevent future bookings (BR-11).
- **Access:** Authorized (Manager)
- **Responses:**
 - 200 OK: Service status updated to inactive.
 - 404 Not Found: Service ID not found.

Module 4: Appointments (/api/appointments)

POST /api/appointments

- **Purpose:** Books an appointment with one or more services.
- **Access:** Authorized (Customer)
- **Request Body:**

```
JSON  
{  
  "vehicleId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",  
  "serviceIds": [  
    "7ca85f64-5717-4562-b3fc-2c963f66afb2"  
  ],  
  "appointmentDate": "2026-10-15T10:00:00Z",  
  "notes": "Slight noise when braking."  
}
```

- **Responses:**
 - 201 Created: Appointment created, snapshot prices saved, background notification queued.
 - 400 Bad Request: Date is in the past (BR-04), or selected service is inactive (BR-03).
 - 403 Forbidden: Vehicle does not belong to the calling customer (BR-02).
 - 409 Conflict: Vehicle already has a booking at the requested time (BR-05).

GET /api/appointments/my

- **Purpose:** Fetches appointment history for the logged-in customer.
- **Access:** Authorized (Customer)
- **Responses:**
 - 200 OK: Array of customer appointments with service snapshots and current statuses.

DELETE /api/appointments/{id}

- **Purpose:** Cancels an upcoming appointment.
- **Access:** Authorized (Customer)
- **Responses:**

- 200 OK: Appointment status set to Cancelled, CancelledAt timestamp set.
- 400 Bad Request: Appointment is already InService or Completed (BR-07).
- 403 Forbidden: Appointment belongs to another customer (BR-06).

PUT /api/appointments/{id}/status

- **Purpose:** Updates appointment operational status.
- **Access:** Authorized (Staff, Manager)
- **Request Body:**
JSON

```
{
  "newStatus": "Confirmed"
}
```
- **Responses:**
 - 200 OK: Status updated successfully.
 - 400 Bad Request: Invalid state transition attempt (BR-08).
 - 403 Forbidden: Non-staff/manager user attempting status modification (BR-09).

Response Standardization

Errors are returned following RFC 7807 (ProblemDetails):

JSON

```
{
  "type": "https://httpstatuses.com/409",
  "title": "Booking Conflict",
  "status": 409,
  "detail": "The vehicle already has an active
appointment during the requested time window."
}
```

7. Non-Functional Requirements (NFRs)

- **Security:**
 - JWT Bearer authentication required for protected endpoints.
 - Role-Based Access Control (RBAC) and claim ownership derived server-side.
 - Passwords hashed using ASP.NET Core Identity default password hashers.
- **Performance & Scalability:**
 - Asynchronous I/O operations throughout controller, service, and data access layers (async/await).
 - Indexing applied to conflict-querying columns.
- **Reliability & Concurrency:**

- Appointment conflict detection and creation must be executed atomically inside a DB transaction to guarantee race condition protection.
- Background job failures in Hangfire must log without rolling back successfully persisted core API transactions.
- **Time Standard:**
 - All database timestamps stored in UTC (DateTime.UtcNow).

8. Database Indexes

To optimize conflict queries and ownership lookups:

- ApplicationUser(Email)
- Vehicle(Vin)
- Vehicle(CustomerId)
- Appointment(CustomerId)
- Appointment(VehicleId, Status, AppointmentDate) (*Composite index for fast overlap checks*)
- AppointmentService(AppointmentId, ServiceId)

9. Acceptance Criteria (Given / When / Then)

Scenario 1: Successful Multi-Service Appointment Booking

- **Given** an authenticated customer who owns Vehicle V-101, and active Services S-1 (\$50) and S-2 (\$100),
- **When** the customer submits a request to book an appointment for V-101 on a future UTC date with services [S-1, S-2],
- **Then** the API creates an Appointment with status Requested, inserts AppointmentService entries retaining snapshot unit prices (\$50, \$100), enqueues a Hangfire notification job for staff, and returns HTTP status 201 Created.

Scenario 2: Rejection of Unauthorized Vehicle Usage

- **Given** Customer A is authenticated, and Vehicle V-999 belongs exclusively to Customer B,
- **When** Customer A attempts to submit an appointment booking using VehicleId = V-999,
- **Then** the API rejects the request, returns HTTP status 403 Forbidden, and creates no database records.

Scenario 3: Attempting Customer Cancellation on In-Service Vehicle

- **Given** an appointment APP-500 associated with Customer A that currently has status InService,
- **When** Customer A sends a DELETE /api/appointments/APP-500 request,
- **Then** the API rejects the request under **BR-07**, returns HTTP status 400 Bad Request with a ProblemDetails payload explaining that in-service appointments cannot be cancelled, and leaves the appointment status as InService.

Scenario 4: Enforcement of State Transition Rules by Staff

- **Given** an appointment APP-200 with status Requested,
- **When** a Staff member attempts to update the status directly to Completed via PUT

/api/appointments/APP-200/status,

- **Then** the API rejects the request under **BR-08**, returns HTTP status 400 Bad Request detailing an invalid state transition path, and keeps the status as Requested.

Scenario 5: Automatic Cancellation of Stale Unconfirmed Requests

- **Given** an appointment APP-100 created with status Requested that has remained unconfirmed for over 24 hours,
- **When** the Hangfire recurring background job executes its sweep interval,
- **Then** the system updates APP-100 status to Cancelled, records CancelledAt = UTC Now, and logs the automated background action without requiring human intervention.

Scenario 6: Historical Price Snapshot Verification

- **Given** Service S-1 has BasePrice \$50 and is booked under Appointment APP-300,
- **When** Manager changes Service S-1 BasePrice to \$75 via PUT /api/services/{id},
- **Then** the existing AppointmentService.UnitPrice for APP-300 remains \$50.

Scenario 7: Service Deactivation Behavior

- **Given** Service S-1 is active when Appointment APP-400 is created,
- **When** Manager deactivates Service S-1 via PATCH /api/services/{id}/deactivate,
- **Then** new appointment creation requests selecting S-1 return 400 Bad Request, while existing Appointment APP-400 remains valid.

Scenario 8: Overlapping Vehicle Booking Conflict

- **Given** Vehicle V-101 has an active appointment spanning 10:00 to 11:00 UTC,
- **When** Customer attempts to book V-101 for a new appointment starting at 10:30 UTC,
- **Then** the API returns 409 Conflict and no new appointment record is persisted.

Scenario 9: Unauthorized Access Rejection

- **Given** a request contains no valid JWT bearer token,
- **When** the client calls GET /api/vehicles/my,
- **Then** the API returns HTTP status 401 Unauthorized.